СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ

Текст програми

ІА/ЛЦ.467200.007 Д4

Київ – 2026 р.

ПЕРЕЛІК ФАЙЛІВ ПРОГРАМНОГО КОМПЛЕКСУ

У цьому додатку наведено повні вихідні тексти ключових модулів програмного комплексу системи трикутного просторового аналізу польоту волейбольного м’яча. Модулі впорядковано відповідно до архітектурного графа залежностей, описаного в розділі 3 пояснювальної записки: спочатку наводиться модуль геометричного зворотного проектування, далі — ядро байєсової фільтрації, потім модуль трекінгу зі скінченним автоматом життєвого циклу треку та, нарешті, сценарій навчання згорткової нейромережі-детектора. Сумарний обсяг наведеного коду становить понад 950 рядків мовою програмування Python.

ball_3d_analysis/get_court_position_methods.py

```
import cv2
import numpy as np
import torch as t
import torchvision
import torchaudio
import albumentations as A
from ultralytics import YOLO


# Reference calibration correspondences: the four court corners.
# Court is 9 m wide x 18 m long; world coordinates are in metres.
pts_real_3d = np.array([
    [0.0, 0.0, 0.0],
```

```
    [9.0, 0.0, 0.0],
    [9.0, 0.0, 18.0],
    [0.0, 0.0, 18.0]],dtype=np.float32)

# Pixel locations of the same four corners in the broadcast frame.
pts_video_2d = np.array([
    [  69, 1033],   # P0  front-left
    [1840, 1031],   # P1  front-right
    [1507,  609],   # P2  back-right
    [ 405,  609]],  # P3  back-left
    dtype=np.float32)

# Initial pinhole intrinsics (principal point at the image centre). The focal
# length is only a starting guess here — it is refined by calibration.
K = np.array([
    [1300.0, 0.0,    960.0],
    [0.0,    1300.0, 540.0],
    [0.0,    0.0,    1.0]],dtype=np.float32)


def calibrate_camera(pts_3d, pts_2d, camera_matrix, dist, up_axis=1):
    ''''''
    Calibrate the camera with PnP. Returns (R, tvec, camera_position):
        R               — world->camera rotation matrix (3x3),
        tvec            — translation into the camera frame (3,1),
        camera_position — camera location in world coordinates (3,1).

    up_axis flips R so the camera ends up above the floor: the four coplanar
```

```
    court corners leave the vertical direction sign-ambiguous otherwise.
    ”””
    success, rvec, tvec = cv2.solvePnP(pts_3d, pts_2d, camera_matrix, dist,
    flags=cv2.SOLVEPNP_ITERATIVE)

    if not success:
        raise ValueError(”solvePnP failed to converge; ”
                         ”check the order of the point correspondences”)

    R, _ = cv2.Rodrigues(rvec)

    # Camera position in world coordinates (C = -R^T * tvec).
    R_inv = np.linalg.inv(R)
    camera_position = -np.dot(R_inv, tvec)

    if (up_axis is not None
            and 0 <= up_axis < 3
            and camera_position[up_axis, 0] < 0.0):
        flip = np.eye(3)
        flip[up_axis, up_axis] = -1.0
        R = np.dot(R, flip)
        camera_position = np.dot(flip, camera_position)

    return R, tvec, camera_position

def get_3d_position(u, v, bbox_w, K, R_matrix, camera_pos,
    ball_diameter=0.21, z_min=2.0, z_max=25.0,
    return_none_on_clip=False):
```

```
’’’’’’
Back-project a ball detection into a 3D world position.

Args:
    u, v:          ball bbox centre, in pixels.
    bbox_w:        ball bbox width, in pixels. Width is used (not height)
                   because it suffers less from motion blur.
    K:             camera intrinsics matrix.
    R_matrix:      rotation matrix from solvePnP.
    camera_pos:    3D camera position (C) from solvePnP.
    ball_diameter: real volleyball diameter, in metres (21 cm nominal).
    z_min, z_max:  sanity bounds on Z_c (distance from camera along the ray).

Returns:
    np.ndarray (3,) — world position [X, Y, Z]; or None when
    return_none_on_clip=True and Z_c fell outside [z_min, z_max].
’’’’’’
f_x = K[0, 0]
c_x = K[0, 2]
f_y = K[1, 1]
c_y = K[1, 2]

Z_c = (f_x * ball_diameter) / bbox_w
if Z_c < z_min or Z_c > z_max:
    if return_none_on_clip:
        return None
    Z_c = max(z_min, min(z_max, Z_c))
```

```
    x_norm = (u - c_x) / f_x
    y_norm = (v - c_y) / f_y
    d_cam = np.array([[x_norm], [y_norm], [1.0]])

    d_world = np.dot(R_matrix.T, d_cam)
    P_world = camera_pos.reshape(3, 1) + Z_c * d_world

    return P_world.ravel()


def get_3d_position_with_cov(u, v, bbox_w, K, R_matrix, camera_pos,
    ball_diameter=0.21, sigma_w=1.0, sigma_uv=1.0, r_floor=0.02,
    z_min=2.0, z_max=25.0, return_none_on_clip=False):
    """Like get_3d_position, but also returns a 3x3 world-frame measurement
    covariance R, anisotropic and consistent with monocular depth geometry.

    Depth error follows from Z_c = f*D / w_box linearised in the bbox width:
        sigma_Zc = |dZ_c/dw| * sigma_w = (f*D / w_box^2) * sigma_w,
    so depth is trusted less for distant (small-w) balls. The uncertainty lies
    along the ray, giving a rank-1 ellipsoid plus lateral pixel terms:
        Cov ~ sigma_Zc^2 * d*d^T + Z_c^2 * sigma_uv^2 * (J_u J_u^T + J_v J_v^T).

    Args:
        sigma_w:  bbox-width sigma, px (detector noise). Default 1.0.
        sigma_uv: bbox-centre sigma in u,v, px. Default 1.0.
        r_floor:  isotropic variance floor (m^2) on every axis. Default 0.02
            (lateral term of the old fixed R), keeping lateral trust stable and
            R positive-definite; only the along-ray depth stays adaptive.
```

```
Returns:
    (P_world (3,), R_cov (3,3)); or (None, None) on clip when
    return_none_on_clip=True.
"""
f_x = K[0, 0]
c_x = K[0, 2]
f_y = K[1, 1]
c_y = K[1, 2]

Z_c = (f_x * ball_diameter) / bbox_w

if Z_c < z_min or Z_c > z_max:
    if return_none_on_clip:
        return None, None
    Z_c = max(z_min, min(z_max, Z_c))

x_norm = (u - c_x) / f_x
y_norm = (v - c_y) / f_y
d_cam = np.array([[x_norm], [y_norm], [1.0]])

d_world = np.dot(R_matrix.T, d_cam)            # (3,1) ray direction
P_world = camera_pos.reshape(3, 1) + Z_c * d_world

# Depth sigma along the ray, from the bbox-width error.
sigma_Zc = (f_x * ball_diameter) / (bbox_w ** 2) * sigma_w

# Lateral Jacobians (point shift per 1 px move of the bbox centre).
```

```
    j_u = np.dot(R_matrix.T, np.array([[1.0 / f_x], [0.0], [0.0]]))
    j_v = np.dot(R_matrix.T, np.array([[0.0], [1.0 / f_y], [0.0]]))

    R_cov = (sigma_Zc ** 2) * np.dot(d_world, d_world.T)
    R_cov += (Z_c ** 2) * (sigma_uv ** 2) * np.dot(j_u, j_u.T)
    R_cov += (Z_c ** 2) * (sigma_uv ** 2) * np.dot(j_v, j_v.T)

    # Isotropic floor: keeps lateral trust at the proven level of the old R
    # (0.02) and guarantees positive-definiteness. Only the along-ray depth
    # stays adaptive (sigma_Zc^2 on top of the floor).
    R_cov += np.eye(3) * r_floor

    return P_world.ravel(), R_cov.astype(np.float64)
```

ball_3d_analysis/IMM_UKF.py

```
import numpy as np
from copy import deepcopy
from scipy.linalg import cholesky, inv, expm, block_diag
from get_court_position_methods import calibrate_camera, get_3d_position
from scipy.stats import multivariate_normal
import sys


class UnscentedKalmanFilter(object):
    def __init__(self, name, dim_x, dim_z, dt, fx, hx, points, sqrt_fn=None,
                 x_mean_fn=None, z_mean_fn=None, residual_x=None, residual_z=None):
```

```
self.name = name
self.x = np.zeros(dim_x)
self.P = np.eye(dim_x)
self.x_prior = np.copy(self.x)
self.P_prior = np.copy(self.P)
self.Q = np.eye(dim_x)
self.R = np.eye(dim_z)
self._dim_x = dim_x
self._dim_z = dim_z
self.points_fn = points
self._dt = dt
self._num_sigmas = points.num_sigmas()
self.hx = hx
self.fx = fx
self.x_mean_fn = x_mean_fn
self.z_mean_fn = z_mean_fn
self.Wm, self.Wc = points.Wm, points.Wc
self.likelihood = None
self._log_likelihood = np.log(sys.float_info.min)
self._mahalanobis = None

if sqrt_fn is None:
    self.msqrt = cholesky
else:
    self.msqrt = sqrt_fn

if residual_x is None:
    self.residual_x = np.subtract
```

```
        else:
            self.residual_x = residual_x

        if residual_z is None:
            self.residual_z = np.subtract
        else:
            self.residual_z = residual_z

        # sigma points transformed through f(x) and h(x)
        self.sigmas_f = np.zeros((self._num_sigmas, self._dim_x))
        self.sigmas_h = np.zeros((self._num_sigmas, self._dim_z))

        self.K = np.zeros((dim_x, dim_z))
        self.y = np.zeros(dim_z)
        self.z = np.array([[None]*dim_z])
        self.S = np.zeros((dim_z, dim_z))
        self.SI = np.zeros((dim_z, dim_z))
        self.inv = np.linalg.inv

        self.x_prior = self.x.copy()
        self.P_prior = self.P.copy()

        self.x_post = self.x.copy()
        self.P_post = self.P.copy()

    def predict(self, dt=None, UT=None, fx=None, **fxargs):
        ''''''
        :param dt: time step for next iteration of computation
```

```
        :param UT: unscented transform function
        :param fx: state transition function
        :param fxargs: arguments passed to f(x)
        ''''''
        if dt is None:
            dt = self._dt

        if UT is None:
            UT = unscented_transform

        # calculating sigma points for given mean and covariance
        self.compute_process_sigmas(dt, fx, **fxargs)
        #pass sigmas through the unscented transform to compute the prior
        self.x, self.P = UT(self.sigmas_f, self.Wm, self.Wc,
                            self.Q, self.x_mean_fn, self.residual_x)
        self.P = (self.P + self.P.T) / 2.0
        self.x_prior = np.copy(self.x)
        self.P_prior = np.copy(self.P)

    def update(self, z, R=None, UT=None, hx=None, **hxargs):
        ''''''
        :param z: measurement vector
        :param R: measurement noise matrix
        :param UT: function for unscented transform
        :param hx: measurement function
        :param hxargs: arguments for hx
        ''''''
        if z is None:
```

```
    self.z = np.array([[None]*self._dim_z]).T
    self.x_post = self.x.copy()
    self.P_post = self.P.copy()
    self.likelihood = 1.0
    return

if hx is None:
    hx = self.hx

if UT is None:
    UT = unscented_transform

if R is None:
    R = self.R
elif np.isscalar(R):
    R = np.eye(self._dim_z) * R

# passing prior sigmas through h(x) to get measurement sigmas
# the shape of simgas_h will vary if the shape of z varies
sigmas_h = []
for s in self.sigmas_f:
    sigmas_h.append(hx(s, **hxargs))

self.sigmas_h = np.atleast_2d(sigmas_h)

# mean and covariance of prediction passed through unscented transform
z_mean, self.S = UT(self.sigmas_h, self.Wm, self.Wc,
                    R, self.z_mean_fn, self.residual_z)
```

```
# Regularise S before inversion / likelihood: guarantees positive
# definiteness even when the covariance is momentarily degenerate.
S_stable = self.S + np.eye(self.S.shape[0]) * 1e-6
try:
    self.SI = np.linalg.inv(S_stable)
except np.linalg.LinAlgError:
    self.SI = np.linalg.pinv(S_stable)

# compute cross variance of the state and measurement
Pxz = self.cross_variance(self.x, z_mean, self.sigmas_f, self.sigmas_h)

self.K = np.dot(Pxz, self.SI) # Kalman gain
self.y = self.residual_z(z, z_mean)

try:
    self.likelihood = multivariate_normal.pdf(
        self.y,
        mean=np.zeros_like(self.y),
        cov=S_stable,
        allow_singular=True,
    )
except (np.linalg.LinAlgError, ValueError):
    self.likelihood = 1e-300

if not np.isfinite(self.likelihood) or self.likelihood <= 0.0:
    self.likelihood = 1e-300

# updating Gaussian state estimate(x, P)
```

```
        self.x = self.x + np.dot(self.K, self.y)
        self.P = self.P - np.dot(self.K, np.dot(self.S, self.K.T))

        # save measurement and posterior state
        self.z = deepcopy(z)
        self.x_post = self.x.copy()
        self.P_post = self.P.copy()

    def cross_variance(self, x, z, sigmas_f, sigmas_h):
        # Computing cross variance of the state 'x' and measurement 'z'
        Pxz = np.zeros((sigmas_f.shape[1], sigmas_h.shape[1]))
        N = sigmas_f.shape[0]
        for i in range(N):
            dx = self.residual_x(sigmas_f[i], x)
            dz = self.residual_z(sigmas_h[i], z)
            Pxz += self.Wc[i] * np.outer(dx, dz)
        return Pxz

    def compute_process_sigmas(self, dt, fx=None, **fx_args):
        # computes the values of sigmas_f
        if fx is None:
            fx = self.fx

        # calculate sigma points for given mean and covariance
        sigmas = self.points_fn.sigma_points(self.x, self.P)

        for i, s in enumerate(sigmas):
            self.sigmas_f[i] = fx(s, dt, **fx_args)
```

```
    @property
    def log_likelihood(self):
        ''''''
        log-likelihood of the last measurement.
        ''''''
        if self._log_likelihood is None:
            self._log_likelihood = logpdf(x=self.y, cov=self.S)
        return self._log_likelihood


    @property
    def mahalanobis(self):
        ''''''''
        Mahalanobis distance of measurement. E.g. 3 means measurement
        was 3 standard deviations away from the predicted value.
        ''''''
        if self._mahalanobis is None:
            self._mahalanobis = np.sqrt(float(np.dot(np.dot(self.y.T, self.SI), self.y)))
        return self._mahalanobis


class IMMEstimator(object):
    '''''' Implements an Interacting Multiple-Model (IMM) estimator.
    :param filters: N-list consisting of filters in sequenced order
    :param mu: (N) array-like of float with mode probabilities for eache filter
    :param M: (N, N) Markov chain transition matrix
    ''''''
```

```
def __init__(self, filters, mu, M):
    if len(filters) < 2:
        raise ValueError('filters must contain at least two filters')

    self.filters = filters
    self.mu = np.asarray(mu) / np.sum(mu)
    self.M = M

    x_shape = filters[0].x.shape
    for f in filters:
        if x_shape != f.x.shape:
            raise ValueError(
                'All filters must have the same state dimension')

    self.x = np.zeros(filters[0].x.shape)
    self.P = np.zeros(filters[0].P.shape)
    self.N = len(filters)
    self.likelihood = np.zeros(self.N)
    self.omega = np.zeros((self.N, self.N))
    self._compute_mixing_probabilities()

    # initialize imm state estimate based on current filters
    self._compute_state_estimate()
    self.x_prior = self.x.copy()
    self.P_prior = self.P.copy()
    self.x_post = self.x.copy()
    self.P_post = self.P.copy()
    self._log_likelihood = np.log(sys.float_info.min)
```

```
        self._mahalanobis = None

    def filter_setx(self):
        for f in self.filters:
            f.x = self.x

    def update(self, z, R=None):
        """
        Add a new measurement (z) to the Kalman filter. If z is None, nothing
        is changed.
        :param z: measurement
        :param R: optional per-detection measurement covariance (3x3). If None,
            each filter uses its own fixed self.R. Passing a per-detection R
            (adaptive sigma_Z) makes depth trust depend on the bbox width.
        """

        # run update on each filter, and save the likelihood
        for i, f in enumerate(self.filters):
            f.update(z, R=R)
            self.likelihood[i] = f.likelihood

        # update mode probabilities from total probability * likelihood
        self.mu = self.cbar * self.likelihood
        sum_mu = np.sum(self.mu)
        if sum_mu == 0.0:
            self.mu = self.cbar.copy()
        else:
            self.mu /= sum_mu
```

```
        self._compute_mixing_probabilities()

        # compute mixed IMM state and covariance and save posterior estimate
        self._compute_state_estimate()
        self.x_post = self.x.copy()
        self.P_post = self.P.copy()

    def predict(self, u=None):
        """
        Predict next state (prior) using the IMM state propagation
        equations.
        :param u: control vector, if None then none
        """

        # compute mixed initial conditions
        xs, Ps = [], []
        for i, (f, w) in enumerate(zip(self.filters, self.omega.T)):
            x = np.zeros(self.x.shape)
            for kf, wj in zip(self.filters, w):
                x += kf.x * wj
            xs.append(x)

            P = np.zeros(self.P.shape)
            for kf, wj in zip(self.filters, w):
                y = kf.x - x
                P += wj * (np.outer(y, y) + kf.P)
            Ps.append(P)
```

```
    #  compute each filter's prior using the mixed initial conditions
    for i, f in enumerate(self.filters):
        # propagate using the mixed state estimate and covariance
        f.x = xs[i].copy()
        f.P = Ps[i].copy()
        f.predict(u)

    # compute mixed IMM state and covariance and save posterior estimate
    self._compute_state_estimate()
    self.x_prior = self.x.copy()
    self.P_prior = self.P.copy()

def _compute_state_estimate(self):
    """
    Computes the IMM's mixed state estimate from each filter using
    the the mode probability self.mu to weight the estimates.
    """
    self.x.fill(0)
    for f, mu in zip(self.filters, self.mu):
        self.x += f.x * mu

    self.P.fill(0)
    for f, mu in zip(self.filters, self.mu):
        y = f.x - self.x
        self.P += mu * (np.outer(y, y) + f.P)

def _compute_mixing_probabilities(self):
```

```
    # Compute the mixing probability for each filter

    self.cbar = np.dot(self.mu, self.M)
    for i in range(self.N):
        for j in range(self.N):
            self.omega[i, j] = (self.M[i, j]*self.mu[i]) / self.cbar[j]

@property
def log_likelihood(self):
    ''''''
    log-likelihood of the last measurement.
    ''''''
    if self._log_likelihood is None:
        self._log_likelihood = logpdf(x=self.y, cov=self.S)
    return self._log_likelihood


@property
def mahalanobis(self):
    ''''''''
    Mahalanobis distance of measurement. E.g. 3 means measurement
    was 3 standard deviations away from the predicted value.
    ''''''
    if self._mahalanobis is None:
        self._mahalanobis = np.sqrt(float(np.dot(np.dot(self.y.T, self.SI), self.y)))
    return self._mahalanobis
```

```
class MerweScaledSigmaPoints(object):

    ,,,,,,
    Generates sigma points and weights according to Van der Merwe's
    2004 dissertation[1] for the UnscentedKalmanFilter class. It
    parametizes the sigma points using alpha, beta, kappa terms, and
    is the version seen in most publications.

    :param n : int
        Dimensionality of the state. 2n+1 weights will be generated.

    :param alpha : float
        Determins the spread of the sigma points around the mean.
        Usually a small positive value (1e-3).

    :param beta : float
        Incorporates prior knowledge of the distribution of the mean. For
        Gaussian x beta=2 is optimal.

    :param kappa : float, default=0.0
        Secondary scaling parameter usually set to 0 or to 3-n.

    :param sqrt_method : function(ndarray), default=scipy.linalg.cholesky
        Defines how we compute the square root of a matrix, which has
        no unique answer. Cholesky is the default choice due to its
        speed(another good choice - scipy.linalg.sqrtm).

    :param subtract : callable (x, y), optional
```

```
    Function that computes the difference between x and y.
    You will have to supply this if your state variable cannot support
    subtraction, such as angles (359-1 degreees is 2, not 358). x and y
    are state vectors, not scalars.

Attributes:
Wm : np.array
    weight for each sigma point for the mean
Wc : np.array
    weight for each sigma point for the covariance
''''''



def __init__(self, n, alpha, beta, kappa, sqrt_method=None, subtract=None):
    #pylint: disable=too-many-arguments

    self.n = n
    self.alpha = alpha
    self.beta = beta
    self.kappa = kappa
    if sqrt_method is None:
        self.sqrt = cholesky
    else:
        self.sqrt = sqrt_method

    if subtract is None:
        self.subtract = np.subtract
    else:
```

```
        self.subtract = subtract

    self._compute_weights()


def num_sigmas(self):
    """ Number of sigma points for each variable in the state x"""
    return 2*self.n + 1


def sigma_points(self, x, P):
    """ Computes the sigma points for an unscented Kalman filter
    given the mean (x) and covariance(P) of the filter.
    Returns tuple of the sigma points and weights.

    Works with both scalar and array inputs:
    sigma_points (5, 9, 2) # mean 5, covariance 9
    sigma_points ([5, 2], 9*eye(2), 2) # means 5 and 2, covariance 9I


    :param x : An array-like object of the means of length n
    :param P : scalar, or np.array
      Covariance of the filter. If scalar, is treated as eye(n)*P.

    :return np.array, of size (n, 2n+1)
       Two dimensional array of sigma points. Each column contains all of
       the sigmas for one dimension in the problem space.
```

```
    Ordered by Xi_0, Xi_{1..n}, Xi_{n+1..2n}
    """

    if self.n != np.size(x):
        raise ValueError("expected size(x) {}, but size is {}".format(
            self.n, np.size(x)))

    n = self.n

    if np.isscalar(x):
        x = np.asarray([x])

    if  np.isscalar(P):
        P = np.eye(n)*P
    else:
        P = np.atleast_2d(P)

    lambda_ = self.alpha**2 * (n + self.kappa) - n
    U = self.sqrt((lambda_ + n)*P)

    sigmas = np.zeros((2*n+1, n))
    sigmas[0] = x
    for k in range(n):
        # pylint: disable=bad-whitespace
        sigmas[k+1]   = self.subtract(x, -U[k])
        sigmas[n+k+1] = self.subtract(x, U[k])

    return sigmas
```

```
    def _compute_weights(self):
        """ Computes the weights for the scaled unscented Kalman filter."""

        n = self.n
        lambda_ = self.alpha**2 * (n +self.kappa) - n

        c = .5 / (n + lambda_)
        self.Wc = np.full(2*n + 1, c)
        self.Wm = np.full(2*n + 1, c)
        self.Wc[0] = lambda_ / (n + lambda_) + (1 - self.alpha**2 + self.beta)
        self.Wm[0] = lambda_ / (n + lambda_)



    def __repr__(self):
        return (f'MerweScaledSigmaPoints(n={self.n}, alpha={self.alpha}, '
                f'beta={self.beta}, kappa={self.kappa})')

def unscented_transform(sigmas, Wm, Wc, noise_cov=None,
                        mean_fn=None, residual_fn=None):
    r"""
    Computes unscented transform of a set of sigma points and weights.
    returns the mean and covariance in a tuple.
    :param residual_fn:
    :param sigmas: ndarray, of size (n, 2n+1)
        2D array of sigma points.
```

```
:param Wm : ndarray [# sigmas per dimension]
    Weights for the mean.


:param Wc : ndarray [# sigmas per dimension]
    Weights for the covariance.

:param noise_cov: ndarray, optional
    noise matrix added to the final computed covariance matrix.

:param mean_fn: callable (sigma_points, weights), optional
    Function that computes the mean of the provided sigma points
    and weights
"""

kmax, n = sigmas.shape

try:
    if mean_fn is None:
        # new mean is just the sum of the sigmas * weight
        x = np.dot(Wm, sigmas)    # dot = \Sigma^n_1 (W[k]*Xi[k])
    else:
        x = mean_fn(sigmas, Wm)
except:
    print(sigmas)
    raise
```

```
    # new covariance is the sum of the outer product of the residuals
    # times the weights

    # this is the fast way to do this - see 'else' for the slow way
    if residual_fn is np.subtract or residual_fn is None:
        y = sigmas - x[np.newaxis, :]
        P = np.dot(y.T, np.dot(np.diag(Wc), y))
    else:
        P = np.zeros((n, n))
        for k in range(kmax):
            y = residual_fn(sigmas[k], x)
            P += Wc[k] * np.outer(y, y)

    if noise_cov is not None:
        P += noise_cov

    return x, P


def Q_discrete_white_noise(dim, dt=1., var=1., block_size=1, order_by_dim=True):
    """
    Returns the Q matrix for the Discrete Constant White Noise
    Model. dim may be either 2, 3, or 4 dt is the time step, and sigma
    is the variance in the noise.

    Q is computed as the G * G^T * variance, where G is the process noise per
    time step. In other words, G = [[.5dt^2][dt]]^T for the constant velocity
```

model.

:param dim : int (2, 3, or 4)
dimension for Q, where the final dimension is (dim x dim)

:param dt : float, default=1.0
time step in whatever units your filter is using for time. i.e. the amount of time between innovations

:param var : float, default=1.0
variance in the noise

:param block_size : int >= 1
If your state variable contains more than one dimension, such as a 3d constant velocity model [x x' y y' z z']^T, then Q must be a block diagonal matrix.

:param order_by_dim : bool, default=True
Defines ordering of variables in the state vector. 'True' orders by keeping all derivatives of each dimensions)
[x x' x'' y y' y'']
whereas 'False' interleaves the dimensions
[x y z x' y' z' x'' y'' z'']
''''''

if not (dim == 2 or dim == 3 or dim == 4):
raise ValueError("dim must be between 2 and 4")

```
    if dim == 2:
        Q = [[.25*dt**4, .5*dt**3],
             [ .5*dt**3,    dt**2]]
    elif dim == 3:
        Q = [[.25*dt**4, .5*dt**3, .5*dt**2],
             [ .5*dt**3,    dt**2,       dt],
             [ .5*dt**2,       dt,        1]]
    else:
        Q = [[(dt**6)/36, (dt**5)/12, (dt**4)/6, (dt**3)/6],
             [(dt**5)/12, (dt**4)/4,  (dt**3)/2, (dt**2)/2],
             [(dt**4)/6,  (dt**3)/2,   dt**2,     dt],
             [(dt**3)/6,  (dt**2)/2 ,  dt,        1.]]


    if order_by_dim:
        return block_diag(*[Q]*block_size) * var
    return order_by_derivative(np.array(Q), dim, block_size) * var


def logpdf(x, mean=None, cov=1, allow_singular=True):
    ''''''
    Computes the log of the probability density function of the normal
    N(mean, cov) for the data x. The normal may be univariate or multivariate.
    ''''''


    if mean is not None:
        flat_mean = np.asarray(mean).flatten()
    else:
        flat_mean = None
```

```
    flat_x = np.asarray(x).flatten()

    return multivariate_normal.logpdf(flat_x, flat_mean,
                                      cov, allow_singular=allow_singular)


def fx_ballistic(x, dt):
    ''''''
    9D ballistic model (CV-CA-IMM): state = [x, vx, ax, y, vy, ay, z, vz, az].

    Constant-acceleration discretisation:
        p_new = p + v*dt + 0.5*a_total*dt^2
        v_new = v + a_total*dt
        a_new = a (deterministic — it only changes through the Q[a] noise).
    ''''''
    vx = np.clip(x[1], -40.0, 40.0)
    vy = np.clip(x[4], -40.0, 40.0)
    vz = np.clip(x[7], -40.0, 40.0)
    ax_s, ay_s, az_s = x[2], x[5], x[8]

    k = 0.0314
    v_mag = np.sqrt(vx**2 + vy**2 + vz**2)
    a_drag_x = -k * v_mag * vx
    a_drag_y = -k * v_mag * vy
    a_drag_z = -k * v_mag * vz

    g = 9.81
    a_tot_x = ax_s + a_drag_x
```

```
    a_tot_y = ay_s + a_drag_y - g
    a_tot_z = az_s + a_drag_z

    return np.array([
        x[0] + vx*dt + 0.5*a_tot_x*dt**2,
        vx + a_tot_x*dt,
        ax_s,
        x[3] + vy*dt + 0.5*a_tot_y*dt**2,
        vy + a_tot_y*dt,
        ay_s,
        x[6] + vz*dt + 0.5*a_tot_z*dt**2,
        vz + a_tot_z*dt,
        az_s,
    ])


def fx_hit(x, dt):
    """9D hit model: identical dynamics to fx_ballistic. The hit mode differs
    only in a much larger Q[a] (set in create_imm_estimator), which lets the
    acceleration wake from ~0 to impulsive values within a frame or two."""
    return fx_ballistic(x, dt)


def fx_bounce(x, dt):
    """
    9D floor-bounce model: vy is inverted with epsilon=0.75, vx/vz are damped
    with friction=0.85.
    """
```

```
    epsilon = 0.75
    friction = 0.85
    x_next = np.copy(x)
    vx_new = x[1] * friction
    vy_new = -x[4] * epsilon
    vz_new = x[7] * friction

    x_next[0] += vx_new * dt
    x_next[3] += vy_new * dt
    x_next[6] += vz_new * dt

    x_next[1] = vx_new
    x_next[4] = vy_new
    x_next[7] = vz_new

    # Reset residual acceleration — a bounce is a ”fresh start”.
    x_next[2] = 0.0
    x_next[5] = 0.0
    x_next[8] = 0.0

    return x_next


def hx(x):
    ”””
    Measurement function 9D -> 3D: extract positions only (indices 0, 3, 6).
    Velocities (1, 4, 7) and accelerations (2, 5, 8) are not measured, only
    estimated by the UKF/IMM.
```

```
    ’’’’’’
    return np.array([x[0], x[3], x[6]])

def measurement_transform(prediction_data, K, R_matrix, camera_pos,
                          ball_diameter=0.21):
    ’’’’’’
    Turn a detection record (px, py, bbox width) into a 3D world measurement
    [X, Y, Z] in court coordinates, with the convention:
        X — across the court (along the end line),
        Y — vertical axis (up, perpendicular to the floor),
        Z — along the long side (along the side line).
    ’’’’’’
    u_cam, v_cam, w_cam = (prediction_data[’x_pos’], prediction_data[’y_pos’],
                           prediction_data[’w_box’])
    raw_x, raw_y, raw_z = get_3d_position(u_cam, v_cam, w_cam, K,
                                          R_matrix, camera_pos, ball_diameter)
    measurement = np.array([raw_x, raw_y, raw_z], dtype=np.float32)

    return measurement

def get_dynamic_transition_matrix(y, vy,
                                  mahalanobis_sq=0.0,
                                  hit_residual_min_sq=8.0,
                                  hit_residual_max_sq=11.34,
                                  bounce_height_max=0.55,
                                  frames_since_update=0,
                                  bounce_max_coast=3,
                                  m_hit_target=None):
```

```
""""""Return the 3x3 IMM Markov transition matrix for [ballistic, hit, bounce]
for the next prediction step. The base matrix sticks to ballistic
(M[0,0]=0.95); two triggers steer it: a passive geometric bounce trigger and
a residual-based mid-air hit trigger interpolated over
[hit_residual_min_sq, hit_residual_max_sq] (the chi^2_3 95th percentile to
the gating bound), clamped outside that range.

y, vy: ball vertical position (m) and velocity (m/s) in world frame.
mahalanobis_sq: squared Mahalanobis distance of the last accepted detection
   (0 for tracks without updates).
""""""
M = np.array([[0.95, 0.04, 0.01],
              [0.60, 0.40, 0.00],
              [0.90, 0.00, 0.10]])

if frames_since_update > bounce_max_coast:
   return np.eye(3)

if (y < bounce_height_max and vy < 0 and frames_since_update
      <= bounce_max_coast):
   M[0] = [0.10, 0.05, 0.85]
   M[1] = [0.10, 0.05, 0.85]
   return M

if mahalanobis_sq > hit_residual_min_sq:
   alpha = min(
      1.0,
      (mahalanobis_sq - hit_residual_min_sq)
```

```
            / max(hit_residual_max_sq - hit_residual_min_sq, 1e-6),
        )
        M_hit_target = (np.array([0.45, 0.50, 0.05])
                        if m_hit_target is None
                        else np.asarray(m_hit_target, dtype=float))
        M[0] = (1.0 - alpha) * M[0] + alpha * M_hit_target

    return M
```

ball_3d_analysis/IMMTracker.py

```
import numpy as np
from scipy.optimize import linear_sum_assignment
from IMM_UKF import *
import cv2



def create_imm_estimator(z_initial, dt=0.02, v_initial=None):
    """Build an IMM estimator from three UKFs (ballistic / hit / bounce).

    z_initial: world position [x, y, z] at init.
    dt:        time step (= 1 / FPS).
    v_initial: optional initial velocity [vx, vy, vz] in m/s. None -> zeros.
               Passing (z_k - z_{k-1}) / dt speeds up UKF convergence and
               avoids residual/Mahalanobis spikes in a new track's first frames.
    """
    # 9D constant-acceleration state: [x, vx, ax, y, vy, ay, z, vz, az].
    dim_x = 9
```

```
dim_z = 3
points = MerweScaledSigmaPoints(n=dim_x, alpha=.1, beta=2., kappa=1.)
# Initial covariance. Accel variance ~25 (m/s^2)^2 lets the filter absorb
# the first non-zero acceleration without immediately blowing the gate;
# velocity variance 50 covers the finite-difference bootstrap spread.
P_init = np.diag([
    0.1, 50.0, 25.0,
    0.1, 50.0, 25.0,
    0.1, 50.0, 25.0,
])
# Measurement noise (m^2). sigma_z ~ 0.7 m reflects monocular depth error
# (Z_c = f*D / bbox_w); tighter values made trajectories too jagged.
R_init = np.diag([0.02, 0.02, 0.5])

# Process noise per model. In Q_discrete_white_noise(dim=3) var is the
# jerk variance (bottom-right block cell), i.e. how much accel may drift.
# ballistic: small jerk noise (free flight is smooth, residual accel ~0).
q_var_ballistic = 0.1
q_b = Q_discrete_white_noise(dim=3, dt=dt, var=q_var_ballistic)
Q_ballistic = block_diag(q_b, q_b, q_b)

# hit: large jerk noise lets accel jump from ~0 to tens of m/s^2 in a frame
# or two. The impulse is modelled through accel growth (Singer-style CA),
# not a raw velocity jump that broke gating in the old 6D version.
q_var_hit = 400
q_h = Q_discrete_white_noise(dim=3, dt=dt, var=q_var_hit)
Q_hit = block_diag(q_h, q_h, q_h)
```

```
# bounce: moderate jerk noise; fx_bounce already resets accel to 0 on
# contact, Q[a] only guards against abrupt deviations in later frames.
q_var_bounce = 5
q_bnc = Q_discrete_white_noise(dim=3, dt=dt, var=q_var_bounce)
Q_bounce = block_diag(q_bnc, q_bnc, q_bnc)

# Ballistic filter
ukf_ballistic = UnscentedKalmanFilter(name='ballistic UKF', dim_x=dim_x,
        dim_z=dim_z, dt=dt, fx=fx_ballistic, hx=hx, points=points)
ukf_ballistic.P = P_init
ukf_ballistic.Q = Q_ballistic
ukf_ballistic.R = R_init

# Hit filter
ukf_hit = UnscentedKalmanFilter(name='hit UKF', dim_x=dim_x, dim_z=dim_z,
                dt=dt, fx=fx_hit, hx=hx, points=points)
ukf_hit.P = P_init
ukf_hit.Q = Q_hit
ukf_hit.R = R_init

# Bounce filter
ukf_bounce = UnscentedKalmanFilter(name='bounce UKF', dim_x=dim_x,
        dim_z=dim_z, dt=dt, fx=fx_bounce, hx=hx, points=points)
ukf_bounce.P = P_init
ukf_bounce.Q = Q_bounce
ukf_bounce.R = R_init

filters_lt = [ukf_ballistic, ukf_hit, ukf_bounce]
```

```
mu = np.array([0.95, 0.04, 0.01])
M_base = np.array([[0.95, 0.04, 0.01],
                   [0.60, 0.40, 0.00],
                   [0.90, 0.00, 0.10]])

imm = IMMEstimator(filters_lt, mu, M_base)

# Initialise the 9D state. Use v_initial if given, else zero velocity.
# Accel always starts at 0 (ballistic prior); the high P_init[a] lets the
# filter discover a non-zero acceleration when needed.
if v_initial is None:
    vx0 = vy0 = vz0 = 0.0
else:
    vx0, vy0, vz0 = (float(v_initial[0]), float(v_initial[1]),
                     float(v_initial[2]))
initial_x = np.array([
    z_initial[0], vx0, 0.0,
    z_initial[1], vy0, 0.0,
    z_initial[2], vz0, 0.0,
])
for f in imm.filters:
    f.x = initial_x.copy()
    # IMMEstimator copies f.x_post into imm.x_post once, in its constructor,
    # before we overwrite f.x. Without re-setting f.x_post here, the birth
    # snapshot logs zeros -> overlay draws a (0,0,0) world point. x_post is
    # only a logging snapshot, so this does not affect predict/update.
    f.x_post = initial_x.copy()
imm._compute_state_estimate()
```

```
    # Same for the IMM aggregate read by snapshot_track in run_segment.
    imm.x_post = initial_x.copy()

    return imm

class Track:
    # Sanity cap on bootstrap velocity (m/s): above this the finite-difference
    # almost certainly linked two different objects, so bootstrap is skipped.
    bootstrap_max_speed = 35.0

    # Track-level covariance reset on a radical residual. A high Mahalanobis^2
    # (~5% tail of chi^2_3) means an impulsive manoeuvre (hit/block); we
    # ”open up” velocity/accel covariance before imm.update(z) so the new
    # measurement pulls the state onto the new trajectory without multi-frame lag.
    cov_reset_mah_sq_threshold = 8.0   # ~95th percentile of chi^2_3
    cov_reset_inflate_v = 50.0         # (m/s)^2 — same level as P_init[v]
    cov_reset_inflate_a = 100.0        # (m/s^2)^2 — 4x P_init[a]

    # Delayed-accept hysteresis. A suspicious detection (mah_sq > threshold) is
    # frozen in _pending and waits for confirmation from later frames: if the
    # next detection is also far, the manoeuvre is real; if it returns to
    # normal, the pending was a false positive and is dropped.
    hysteresis_mah_sq_threshold = 8.0   # suspicion threshold (= cov_reset)
    hysteresis_confirm_threshold = 5.0  # next mah_sq below this -> pending was FP
    hysteresis_window = 6               # max frames to wait for confirmation
    coast_threshold = 4                 # consecutive misses = coasting: high
                                        # mah_sq is expected (P grew naturally),
                                        # not a manoeuvre -> no inflate, no defer
```

```
# Coast covariance cap. While coasting P grows unbounded; large P_vel
# spreads the UKF sigma points so wide that the quadratic drag (Jensen)
# pulls the sigma-mean velocity toward zero and overrides gravity -> vy
# freezes and the track ”hangs” instead of following a ballistic arc. The
# cap keeps sigma points tight so gravity dominates again. pos cap = 9 m^2
# (+-3 m) matches Gate A so the Mahalanobis gate stays meaningful.
coast_p_pos_cap = 9.0    # (m^2) — +-3 m, = Gate A max_assoc_residual
coast_p_vel_cap = 50.0   # (m/s)^2 — = P_init[v]
coast_p_acc_cap = 100.0  # (m/s^2)^2 — = cov_reset_inflate_a

def __init__(self, track_id, z_initial, dt, v_initial=None,
             initial_hits=1, min_hits=3, enable_hysteresis=False,
             enable_cov_reset=False, enable_coast_cov_cap=False,
             enable_coast_z_fence=False, coast_z_min=-3.0, coast_z_max=21.0,
             hit_residual_min_sq=8.0, hit_residual_max_sq=11.34,
             bounce_height_max=0.55, bounce_max_coast=3,
             m_hit_target=None):
    ”””
    enable_cov_reset: inflate P[v,a] when mah_sq > threshold. Default False
                      (lets the residual live 1-2 frames so the hit trigger
                      fires and IMM likelihood raises mu_hit; cov-reset and
                      hit are substitutes for the same residual signal).
    enable_hysteresis: delayed-accept defer of suspicious detections. Default
                      False (defer cannibalises cov-reset and the hit trigger
                      — all three share mah_sq=8 — and zeroes the residual
                      the next predict needs). Coast-skip via _missed_streak
                      stays active regardless: that handles occlusion, not defer.
```

v_initial: optional initial velocity vector; lets the UKFs start
with a real velocity instead of zeros.
initial_hits: hits the track starts with. Bootstrapped tracks use 2
(two consecutive detections were effectively seen),
promoting to Confirmed one frame sooner.
min_hits: updates needed to reach 'Confirmed'. Forwarded from
IMMTracker (CLI) so the swept parameter affects the
track state machine.
"""

```
self.track_id = track_id
self.dt = float(dt)
self.enable_hysteresis = bool(enable_hysteresis)
self.enable_cov_reset = bool(enable_cov_reset)
self.enable_coast_cov_cap = bool(enable_coast_cov_cap)
# Coast Z-fence: while coasting there is no measurement, so no gate acts
# and predict() extrapolates world depth Z (idx 6) from a contaminated
# depth velocity vz (idx 7). Near the frame edge w_box collapses, Z=f*D/w
# inflates, and coasting carries the ball to Z=33-42 m (court is 18 m).
# The fence clamps predicted Z to [coast_z_min, coast_z_max] and damps
# vz/az that push past it. Active only during coasting (mark_missed).
self.enable_coast_z_fence = bool(enable_coast_z_fence)
self.coast_z_min = float(coast_z_min)
self.coast_z_max = float(coast_z_max)
# IMM triggers, forwarded to get_dynamic_transition_matrix on each
# predict(). Defaults match the function signature (A/B compatibility).
self.hit_residual_min_sq = float(hit_residual_min_sq)
self.hit_residual_max_sq = float(hit_residual_max_sq)
self.bounce_height_max = float(bounce_height_max)
```

```
self.bounce_max_coast = int(bounce_max_coast)
self.m_hit_target = (None if m_hit_target is None
                     else np.asarray(m_hit_target, dtype=float))
self.imm = create_imm_estimator(z_initial, dt, v_initial=v_initial)

# Remember the first detection for intra-track velocity bootstrap on the
# second hit. Skip it if v_initial was already supplied externally.
self.z_initial = np.asarray(z_initial, dtype=float).copy()
self._velocity_bootstrapped = (v_initial is not None)

# Lifecycle
self.state = 'Tentative'  # 'Tentative', 'Confirmed', 'Deleted'
self.time_since_update = 0
self.hits = int(initial_hits)
self.hit_streak = int(initial_hits)
self.min_hits = int(min_hits)
# Last accepted Mahalanobis^2 (gating residual), passed to
# get_dynamic_transition_matrix to activate the hit trigger on the next
# predict(). Reset to 0 after frames without updates.
self._last_mahalanobis_sq = 0.0
# Hysteresis state
self._pending = None              # Optional[(z, mah_sq)]
self._pending_frames_elapsed = 0  # frames since pending was set
self._missed_streak = 0           # consecutive mark_missed (coast detect)
# Position-extraction matrix: indices 0, 3, 6 of the 9D state. Velocities
# and accelerations are not measured, only estimated by the UKF.
self.h_matrix = np.array([
    [1, 0, 0, 0, 0, 0, 0, 0, 0],   # x
```

```
        [0, 0, 0, 1, 0, 0, 0, 0, 0],   # y
        [0, 0, 0, 0, 0, 0, 1, 0, 0],   # z
    ], dtype=float)

def _inflate_covariance(self):
    """Inflate P[v,a] for every sub-filter and the IMM aggregate."""
    for f in self.imm.filters:
        f.P[1, 1] += self.cov_reset_inflate_v
        f.P[4, 4] += self.cov_reset_inflate_v
        f.P[7, 7] += self.cov_reset_inflate_v
        f.P[2, 2] += self.cov_reset_inflate_a
        f.P[5, 5] += self.cov_reset_inflate_a
        f.P[8, 8] += self.cov_reset_inflate_a
    self.imm.P[1, 1] += self.cov_reset_inflate_v
    self.imm.P[4, 4] += self.cov_reset_inflate_v
    self.imm.P[7, 7] += self.cov_reset_inflate_v
    self.imm.P[2, 2] += self.cov_reset_inflate_a
    self.imm.P[5, 5] += self.cov_reset_inflate_a
    self.imm.P[8, 8] += self.cov_reset_inflate_a

def _collapse_accel_to_ballistic(self):
    """Zero the residual acceleration (indices 2, 5, 8) in all states.

    Called while coasting (mark_missed). Without a measurement, predict()
    keeps integrating stale accel into velocity (vx += ax*dt) and the track
    accelerates across the court. Physically the ball between contacts has
    zero residual accel (only g + drag, already driving terms in fx_*), so
    collapsing to ballistic keeps coasting at constant velocity. Velocity,
```

```
        position and P are left untouched.
        """
        for f in self.imm.filters:
            f.x[2] = f.x[5] = f.x[8] = 0.0
            f.x_post[2] = f.x_post[5] = f.x_post[8] = 0.0
        self.imm.x[2] = self.imm.x[5] = self.imm.x[8] = 0.0
        self.imm.x_post[2] = self.imm.x_post[5] = self.imm.x_post[8] = 0.0

    @staticmethod
    def _cap_P_matrix(P, caps):
        """PD-safe cap of the P diagonal to caps[i]. For each i with
        P[i,i] > caps[i] scale row and column i by s = sqrt(cap/P[i,i]). This
        congruence transform D*P*D preserves positive-definiteness (unlike a
        naive P[i,i]=cap, which can break PD and fail the UKF Cholesky); the
        off-diagonals scale proportionally, keeping correlations valid."""
        for i in range(P.shape[0]):
            if P[i, i] > caps[i]:
                s = np.sqrt(caps[i] / P[i, i])
                P[i, :] *= s
                P[:, i] *= s

    def _cap_coast_covariance(self):
        """Cap P-diagonal growth while coasting (see coast_p_*_cap). Without it
        P grows unbounded, the UKF sigma points spread, the quadratic drag damps
        gravity in the sigma-mean, vy freezes and the track hangs instead of
        following a ballistic arc. Complements Gate A."""
        caps = np.array([
            self.coast_p_pos_cap, self.coast_p_vel_cap, self.coast_p_acc_cap,
```

```
            self.coast_p_pos_cap, self.coast_p_vel_cap, self.coast_p_acc_cap,
            self.coast_p_pos_cap, self.coast_p_vel_cap, self.coast_p_acc_cap,
        ])
        for f in self.imm.filters:
            self._cap_P_matrix(f.P, caps)
        self._cap_P_matrix(self.imm.P, caps)

    def _fence_coast_depth(self):
        """Clamp world depth Z (idx 6) while coasting to [coast_z_min,
        coast_z_max] and damp the depth velocity/accel (vz=7, az=8) that push
        past the fence. Applied to all UKF filters (their x and x_prior seed the
        next sigma points) and to the mixed imm state (reported in overlay).
        P is left to Gate C (coast_cov_cap)."""
        zlo, zhi = self.coast_z_min, self.coast_z_max

        def _clamp(x):
            if x[6] > zhi:
                x[6] = zhi
                if x[7] > 0.0:
                    x[7] = 0.0
                    x[8] = 0.0
            elif x[6] < zlo:
                x[6] = zlo
                if x[7] < 0.0:
                    x[7] = 0.0
                    x[8] = 0.0

        for f in self.imm.filters:
```

```
            _clamp(f.x)
            _clamp(f.x_prior)
        _clamp(self.imm.x)
        _clamp(self.imm.x_prior)

    def predict(self):
        # 9D state: y = x[3], vy = x[4].
        self.imm.M = get_dynamic_transition_matrix(
            self.imm.x[3], self.imm.x[4],
            mahalanobis_sq=self._last_mahalanobis_sq,
            frames_since_update=self.time_since_update,
            hit_residual_min_sq=self.hit_residual_min_sq,
            hit_residual_max_sq=self.hit_residual_max_sq,
            bounce_height_max=self.bounce_height_max,
            bounce_max_coast=self.bounce_max_coast,
            m_hit_target=self.m_hit_target,
        )
        self.imm.predict()
        if self._pending is not None:
            self._pending_frames_elapsed += 1
            if self._pending_frames_elapsed > self.hysteresis_window:
                # Confirmation window exhausted: drop pending, resume ageing.
                self._pending = None
                self._pending_frames_elapsed = 0
                self.time_since_update += 1
            # else: track does not age during an active hysteresis window
        else:
            self.time_since_update += 1
```

```
def update(self, z, R=None):
    # R: optional per-detection measurement covariance (adaptive sigma_Z).
    # None -> the IMM filters use their own fixed self.R.
    current_mah_sq = self._last_mahalanobis_sq

    # Delayed-accept hysteresis. Defer (pending) runs only with
    # enable_hysteresis. Coast-skip (via _missed_streak) is always active:
    # it handles return from occlusion, not detection deferral.
    _coast_skip = False  # True -> skip inflate and the hysteresis trigger

    if self.enable_hysteresis and self._pending is not None:
        pending_z, _ = self._pending
        self._pending = None
        self._pending_frames_elapsed = 0

        if current_mah_sq >= self.hysteresis_confirm_threshold:
            # Both frames far from the prediction -> real manoeuvre. Inflate
            # P and update with the current z. pending_z (the late
            # measurement) is not applied: a second imm.update() without a
            # predict() between them breaks PD of P (Cholesky fail).
            self._inflate_covariance()
            self.imm.update(z, R=R)
            self.time_since_update = 0
            self.hits += 1
            self.hit_streak += 1
            if self.state == 'Tentative' and self.hits >= self.min_hits:
                self.state = 'Confirmed'
```

```
            self._missed_streak = 0
            return

        # mah_sq < confirm threshold -> the detection returned to a normal
        # trajectory -> pending was a false positive. Continue with a normal
        # update, no inflate and no new trigger.
        _coast_skip = True
        self._missed_streak = 0

    elif self._missed_streak >= self.coast_threshold:
        # Long miss streak -> high mah_sq is expected (P grew naturally while
        # coasting), not a manoeuvre. Skip inflate and the hysteresis trigger.
        _coast_skip = True
        self._missed_streak = 0

    elif (self.enable_hysteresis and z is not None
          and current_mah_sq > self.hysteresis_mah_sq_threshold):
        # Suspicious detection: defer it for confirmation.
        self._pending = (np.asarray(z, dtype=float).copy(), current_mah_sq)
        self._pending_frames_elapsed = 0
        self.imm.update(None)   # extrapolate instead of accepting
        self.hit_streak = 0
        self._last_mahalanobis_sq = 0.0
        self._missed_streak += 1
        return

    # Mechanism A — intra-track velocity bootstrap.
    # On the hits=1 -> 2 transition the UKFs still had v=0 in the prior,
```

```
# which inflates the residual/Mahalanobis and makes the Kalman gain pull
# velocity to truth slowly. We compute v_bootstrap from the finite
# difference (z - z_initial)/(gap*dt) and write it into every filter
# before imm.update, so predict/update run with a correct velocity prior
# from the very first use.
do_bootstrap = (
    z is not None
    and self.hits == 1
    and not self._velocity_bootstrapped
    and self.time_since_update > 0
)
if do_bootstrap:
    gap_dt = self.time_since_update * self.dt
    z_arr = np.asarray(z, dtype=float)
    v_boot = (z_arr - self.z_initial) / gap_dt
    speed = float(np.linalg.norm(v_boot))
    if 0.0 < speed <= self.bootstrap_max_speed:
        # Write velocity into all sub-filters and their x_prior (which
        # seeds the next predict sigma points). Accel (2, 5, 8) is left
        # at 0 — there is no basis to bootstrap it. Velocity indices are
        # [1, 4, 7] in the 9D state.
        for f in self.imm.filters:
            f.x[1] = v_boot[0]
            f.x[4] = v_boot[1]
            f.x[7] = v_boot[2]
            f.x_prior[1] = v_boot[0]
            f.x_prior[4] = v_boot[1]
            f.x_prior[7] = v_boot[2]
```

```
        # Update the mixed state too.
        self.imm.x[1] = v_boot[0]
        self.imm.x[4] = v_boot[1]
        self.imm.x[7] = v_boot[2]
        self.imm.x_prior[1] = v_boot[0]
        self.imm.x_prior[4] = v_boot[1]
        self.imm.x_prior[7] = v_boot[2]
        self._velocity_bootstrapped = True

# Step 2.A — track-level covariance reset on a radical residual. Runs
# after the bootstrap (the bootstrap frame is not a hit) but before
# imm.update(z). _coast_skip means the high mah_sq is expected, so no
# inflate is needed.
do_cov_reset = (
    self.enable_cov_reset
    and z is not None
    and not _coast_skip
    and self._last_mahalanobis_sq > self.cov_reset_mah_sq_threshold
)
if do_cov_reset:
    self._inflate_covariance()

self.imm.update(z, R=R)
self.time_since_update = 0
self.hits += 1
self.hit_streak += 1
if self.state == 'Tentative' and self.hits >= self.min_hits:
    self.state = 'Confirmed'
```

```
        self._missed_streak = 0

    def mark_missed(self):
        self.hit_streak = 0
        # No measurement -> residual undefined. Reset to 0 so the next predict()
        # does not fire the hit trigger from stale data.
        self._last_mahalanobis_sq = 0.0
        self._missed_streak += 1
        self.imm.update(None)     # extrapolate without a measurement
        # Damp residual accel so coasting does not accelerate by integrating
        # stale ax into velocity.
        self._collapse_accel_to_ballistic()
        # Gate C: cap P-diagonal growth while coasting (keeps extrapolation
        # physical past the frame edge).
        if self.enable_coast_cov_cap:
            self._cap_coast_covariance()
        # Coast Z-fence: keep extrapolated depth physical (<= court size).
        if self.enable_coast_z_fence:
            self._fence_coast_depth()

    def get_mahalanobis_distance(self, z, R_matrix):
        """Mahalanobis distance between the IMM prediction and a new detection."""
        # Project the mixed prior state x_prior and covariance P_prior into
        # measurement space.
        z_mean = np.dot(self.h_matrix, self.imm.x_prior)
        S = np.dot(self.h_matrix, np.dot(self.imm.P_prior,
                    self.h_matrix.T)) + R_matrix
```

```
        y = z - z_mean  # innovation

        try:
            S_inv = np.linalg.inv(S)
            dist_sq = np.dot(y.T, np.dot(S_inv, y))
            return dist_sq
        except np.linalg.LinAlgError:
            return 1e5


class IMMTracker:
    def __init__(self, dt=0.02, max_age=80, min_hits=3, gating_threshold=16.0,
                 bootstrap_max_gap_frames=5, bootstrap_max_dist=2.5,
                 bootstrap_max_speed=35.0, enable_hysteresis=False,
                 enable_cov_reset=False, enable_single_ball_nms=True,
                 enable_physical_gate=True, max_assoc_residual=3.0,
                 enable_coast_cov_cap=False, spawn_suppress_max_coast=0,
                 enable_depth_robust_gate=False, depth_hold_gain=0.5,
                 depth_innov_free=0.0,
                 enable_coast_z_fence=False, coast_z_min=-3.0, coast_z_max=21.0,
                 tentative_max_age=5, floor_kill_y=-0.3,
                 enable_adaptive_depth_R=False,
                 hit_residual_min_sq=8.0, hit_residual_max_sq=11.34,
                 bounce_height_max=0.55, bounce_max_coast=3,
                 m_hit_target=None):
        ''''''
        dt:               time step (= 1 / FPS).
        max_age:          frames without an update before a track is deleted.
```

min_hits: frames to confirm a track (Tentative -> Confirmed).

gating_threshold: Mahalanobis chi^2 gate. chi^2_3 p=0.99 = 11.34; 16.0 admits borderline manoeuvre detections (mah 12-15) to revive the hit mode without growth.

bootstrap_max_gap_frames: max gap (frames) between a pending detection and the current one to bootstrap velocity. Default 5.

bootstrap_max_dist: max 3D distance (m) between pending and current detection for bootstrap. Default 2.5 m.

bootstrap_max_speed: sanity cap (m/s); above it bootstrap is skipped (likely a false link of two objects). Default 35.

enable_hysteresis: delayed-accept hysteresis per Track. Default False (defer cannibalises cov-reset + hit).

enable_cov_reset: Step 2.A covariance reset per Track. Default False (Path B) to keep the IMM hit mode alive.

enable_single_ball_nms: one ball -> at most one Confirmed track. Extra confirmed tracks (phantom duplicates) are removed, keeping the best. Default True.

enable_physical_gate: Gate A. A covariance-independent ceiling on the Euclidean residual ||z - prediction||. The Mahalanobis gate depends on P, which inflates while coasting and lets an 8 m FP jump pass; a fixed ceiling does not. Default True.

max_assoc_residual: Gate A threshold (m). Real-manoeuvre residuals reach ~2.2 m; the nearest teleport jumps to ~6 m, so 3.0 m cleanly separates them.

”””

self.dt = dt

self.max_age = max_age

```
self.min_hits = min_hits
self.gating_threshold = gating_threshold

self.bootstrap_max_gap_frames = int(bootstrap_max_gap_frames)
self.bootstrap_max_dist = float(bootstrap_max_dist)
self.bootstrap_max_speed = float(bootstrap_max_speed)
self.enable_hysteresis = bool(enable_hysteresis)
self.enable_cov_reset = bool(enable_cov_reset)
self.enable_single_ball_nms = bool(enable_single_ball_nms)
# Gate A: covariance-independent physical ceiling on the residual.
self.enable_physical_gate = bool(enable_physical_gate)
self.max_assoc_residual = float(max_assoc_residual)
# Gate C: cap P-diagonal growth while coasting (forwarded to Track).
self.enable_coast_cov_cap = bool(enable_coast_cov_cap)
# Gate D: suppress spawning a parasite track while the real Confirmed
# track is briefly coasting (0 < tsu <= spawn_suppress_max_coast). One
# ball domain: a jumpy occluded detection used to spawn a competitor that
# then stole identity. 0 = disabled (A/B compatibility); ~8 covers a
# typical hand occlusion at 50 FPS.
self.spawn_suppress_max_coast = int(spawn_suppress_max_coast)
# Gate A-depth: depth-robust Gate A. Split the residual into lateral
# (X, Y — the pixel projection YOLO gives correctly even under occlusion)
# and depth (Z — monocular f*D/w, which degrades as w collapses). Gate
# only the lateral part hard (hypot(dX,dY) <= max_assoc_residual) and,
# instead of rejecting a large depth innovation, strongly distrust it by
# inflating sigma_Z^2. Depth is then held by the prediction while X,Y
# update from the valid detection. Default False (A/B compatibility).
self.enable_depth_robust_gate = bool(enable_depth_robust_gate)
```

```
self.depth_hold_gain = float(depth_hold_gain)
# Huber-style dead zone for depth distrust: damp only the excess of
# |innov| over a normal-physics threshold. excess = max(0, |innov| -
# depth_innov_free); sigma_Z^2 += (gain*excess)^2. Pure (gain*innov)^2
# also damped legitimate depth (far side, serve); ~3 m dead zone passes
# normal physics and suppresses only occlusion outliers (6-11 m).
# 0.0 = disabled. Recommended ~3.0 (= max_assoc_residual).
self.depth_innov_free = float(depth_innov_free)
# Coast Z-fence: while coasting no gate acts and predict() extrapolates
# world Z from a contaminated vz (w_box collapses near the frame edge ->
# Z=f*D/w inflates -> vz up to +-19 m/s, Z carried to 33-42 m on an 18 m
# court). The fence in each track's mark_missed clamps predicted Z to
# [coast_z_min, coast_z_max] and damps vz/az. Default False. Recommended
# coast_z_max~21 (back line 18 m + ~3 m for servers), coast_z_min~-3.
self.enable_coast_z_fence = bool(enable_coast_z_fence)
self.coast_z_min = float(coast_z_min)
self.coast_z_max = float(coast_z_max)
# Defect 1: ghost tracks. A Tentative track that never reached min_hits
# must not coast as a full track — 1-2 noise detections otherwise draw
# an ~80-frame fiction. Kill Tentative tracks after this many coast frames.
self.tentative_max_age = int(tentative_max_age)
# Defect 2: sub-floor coasting. A ball in flight cannot be below the
# floor; if a coasting track extrapolates y < floor_kill_y, terminate it.
# Small negative margin against floor noise (ball r ~0.1 m).
self.floor_kill_y = float(floor_kill_y)
# Adaptive sigma_Z. If True and detection_covs is passed to update(),
# gating and the IMM update use per-detection 3x3 R (anisotropic, along
# the ray; sigma_Zc = f*D/w^2 * sigma_w) instead of the fixed
```

```
# self.R_matrix. Default False (A/B compatibility).
self.enable_adaptive_depth_R = bool(enable_adaptive_depth_R)
# IMM triggers, forwarded to every Track -> get_dynamic_transition_matrix.
# Allow tuning bounce/hit activation from the CLI. Defaults match the
# function signature (A/B compatibility).
self.hit_residual_min_sq = float(hit_residual_min_sq)
self.hit_residual_max_sq = float(hit_residual_max_sq)
self.bounce_height_max = float(bounce_height_max)
self.bounce_max_coast = int(bounce_max_coast)
self.m_hit_target = (None if m_hit_target is None
                     else np.asarray(m_hit_target, dtype=float))
# Gate B: merge-identity guard. Two Confirmed tracks inherit a shared id
# only if physically reconcilable (same ball split by a manoeuvre, within
# this distance). A distant FP track that reached Confirmed does not
# donate its id to the real coasting track (that would amplify teleports).
# Manoeuvre fragments stay <=~2 m, FP teleports >=6 m, so 4.0 m separates.
self.single_ball_merge_max_dist = 4.0
# Coasting <= this many frames does not cost identity (we bucket tsu).
# tol=1: a real track survives one missed frame (detection noise) but
# after 2+ coast frames yields to an actively-detected track. A small tol
# minimises the spatial jump at a switch.
self.single_ball_nms_tsu_tol = 1

self.tracks = []
self.next_id = 1
# Matrix for Mahalanobis gating, consistent with R_init in
# create_imm_estimator (sigma_z ~ 0.7 m for monocular depth).
self.R_matrix = np.diag([0.02, 0.02, 0.5])
```

```
        # Ring buffer of unassigned detections from the last few frames. Used to
        # bootstrap velocity when spawning a new track: if the current
        # unassigned detection is close to one from the last N frames, init the
        # track with v = (z_now - z_prev)/(dframe*dt) instead of zeros.
        self._spawn_buffer = []  # elements: (np.array z, int frame_idx)
        self._frame_counter = 0

    def update(self, detections_3d, detection_covs=None):
        """
        detections_3d: list of np.array([x, y, z]) for all detected objects
            in the frame.
        detection_covs: optional parallel list of 3x3 measurement covariances
            (one per detection). Used only if enable_adaptive_depth_R=True;
            None -> fixed self.R_matrix.
        """
        self._frame_counter += 1

        def _R_for(d_idx):
            """R for detection d: adaptive if enabled and provided."""
            if (self.enable_adaptive_depth_R
                    and detection_covs is not None
                    and detection_covs[d_idx] is not None):
                return detection_covs[d_idx]
            return self.R_matrix

        def _depth_robust_R(track, z, R_base):
            """Inflate sigma_Z^2 proportional to the squared depth innovation so
```

```
        a large depth jump barely moves the filter (depth held by the
        prediction, X,Y updated from the valid detection). z_pred from
        x_prior (predict already ran for all tracks).""""
        z_pred = np.dot(track.h_matrix, track.imm.x_prior)
        depth_innov = float(z[2] - z_pred[2])
        # Dead zone: damp only the excess of |innov| over normal physics.
        excess = max(0.0, abs(depth_innov) - self.depth_innov_free)
        R_eff = np.array(R_base, dtype=float, copy=True)
        R_eff[2, 2] += (self.depth_hold_gain * excess) ** 2
        return R_eff

    for track in self.tracks:
        track.predict()

    if len(detections_3d) == 0:
        for track in self.tracks:
            track.mark_missed()
        self._age_spawn_buffer()
        self._manage_lifecycle()
        return

    if len(self.tracks) == 0:
        # No tracks yet. Any detection may bootstrap if the spawn buffer has
        # a predecessor.
        for z in detections_3d:
            self._spawn_track_from_unmatched(z)
        self._age_spawn_buffer()
        return
```

```
cost_matrix = np.full((len(self.tracks), len(detections_3d)), 1e5)

for t, track in enumerate(self.tracks):
    # Gate A reference point: the track's predicted position h*x_prior.
    z_pred = np.dot(track.h_matrix, track.imm.x_prior)
    for d, z in enumerate(detections_3d):
        R_d = _R_for(d)
        # Gate A: reject a pair whose Euclidean jump from the prediction
        # is physically impossible in one step, independent of how much P
        # inflated while coasting (that is what opened the Mahalanobis
        # gate for teleports).
        if self.enable_physical_gate:
            if self.enable_depth_robust_gate:
                # Gate A-depth: gate only the lateral (X,Y) part. The
                # large depth innovation is not rejected but handled by
                # depth distrust (inflated sigma_Z in gating and update),
                # so a detection with valid u,v is accepted.
                lateral = float(np.hypot(z[0] - z_pred[0],
                                         z[1] - z_pred[1]))
                if lateral > self.max_assoc_residual:
                    continue
                R_d = _depth_robust_R(track, z, R_d)
            elif np.linalg.norm(z - z_pred) > self.max_assoc_residual:
                continue
        dist_sq = track.get_mahalanobis_distance(z, R_d)
        if dist_sq < self.gating_threshold:
            cost_matrix[t, d] = dist_sq
```

```
# Association (Hungarian algorithm).
row_ind, col_ind = linear_sum_assignment(cost_matrix)

unmatched_tracks = set(range(len(self.tracks)))
unmatched_detections = set(range(len(detections_3d)))

# Update matched pairs.
for r, c in zip(row_ind, col_ind):
    if cost_matrix[r, c] < self.gating_threshold:
        # Save Mahalanobis^2 at accept time; it feeds
        # get_dynamic_transition_matrix on the next predict().
        self.tracks[r]._last_mahalanobis_sq = float(
            cost_matrix[r, c]
        )
        R_c = _R_for(c)
        if self.enable_depth_robust_gate:
            # Same inflated sigma_Z as in gating: depth held by the
            # prediction, X,Y updated from the detection.
            R_c = _depth_robust_R(self.tracks[r],
                                  detections_3d[c], R_c)
        self.tracks[r].update(detections_3d[c], R=R_c)
        unmatched_tracks.remove(r)
        unmatched_detections.remove(c)

# Missed tracks.
for t in unmatched_tracks:
    self.tracks[t].mark_missed()
```

```
    # Spawn new tracks for unmatched detections (with velocity bootstrap if a
    # predecessor is found). Gate D: if a real Confirmed track is briefly
    # coasting (0 < tsu <= spawn_suppress_max_coast), do not spawn a
    # competitor from outlier detections — and keep the outlier out of the
    # spawn buffer so a later legitimate spawn starts with zero velocity.
    suppress_spawn = (
        self.spawn_suppress_max_coast > 0
        and any(
            t.state == 'Confirmed'
            and 0 < t.time_since_update <= self.spawn_suppress_max_coast
            for t in self.tracks
        )
    )
    if not suppress_spawn:
        for d in unmatched_detections:
            self._spawn_track_from_unmatched(detections_3d[d])

    # Age the spawn buffer and remove stale tracks.
    self._age_spawn_buffer()
    self._manage_lifecycle()


# ------------------------------------------------------------
# Initial-velocity bootstrap helpers
# ------------------------------------------------------------
def _spawn_track_from_unmatched(self, z_now):
    """Create a new Track for the unassigned detection z_now. If the spawn
    buffer holds a predecessor that satisfies the constraints (close in
```

```
space, recent in time, consistent velocity), init the track with
v_initial = (z_now - z_prev)/(dframe*dt) and hits=2; otherwise a plain
zero-velocity init. The used spawn point is removed (one-to-one) to avoid
gluing two tracks to the same historical detection.""""
z_now_arr = np.asarray(z_now, dtype=float)
best_idx = -1
best_dist = self.bootstrap_max_dist
best_v = None
best_gap = 0

for i, (z_prev, f_prev) in enumerate(self._spawn_buffer):
    gap = self._frame_counter - f_prev
    if gap <= 0 or gap > self.bootstrap_max_gap_frames:
        continue
    d_xyz = float(np.linalg.norm(z_now_arr - z_prev))
    if d_xyz >= best_dist:
        continue
    gap_dt = gap * self.dt
    if gap_dt <= 0:
        continue
    v_cand = (z_now_arr - z_prev) / gap_dt
    speed = float(np.linalg.norm(v_cand))
    if speed > self.bootstrap_max_speed:
        # Velocity beyond physically possible -> no bootstrap.
        continue
    best_idx = i
    best_dist = d_xyz
    best_v = v_cand
```

```
            best_gap = gap

    if best_idx >= 0:
        # Bootstrap: create a track with non-zero v and hits=2.
        self.tracks.append(
            Track(self.next_id, z_now_arr, self.dt,
                  v_initial=best_v, initial_hits=2,
                  min_hits=self.min_hits,
                  enable_hysteresis=self.enable_hysteresis,
                  enable_cov_reset=self.enable_cov_reset,
                  enable_coast_cov_cap=self.enable_coast_cov_cap,
                  enable_coast_z_fence=self.enable_coast_z_fence,
                  coast_z_min=self.coast_z_min,
                  coast_z_max=self.coast_z_max,
                  hit_residual_min_sq=self.hit_residual_min_sq,
                  hit_residual_max_sq=self.hit_residual_max_sq,
                  bounce_height_max=self.bounce_height_max,
                  bounce_max_coast=self.bounce_max_coast,
                  m_hit_target=self.m_hit_target)
        )
        self.next_id += 1
        # Remove the used spawn point.
        del self._spawn_buffer[best_idx]
    else:
        # Plain zero-velocity spawn + buffer the detection for a future
        # bootstrap.
        self.tracks.append(
            Track(self.next_id, z_now_arr, self.dt,
```

```
                min_hits=self.min_hits,
                enable_hysteresis=self.enable_hysteresis,
                enable_cov_reset=self.enable_cov_reset,
                enable_coast_cov_cap=self.enable_coast_cov_cap,
                enable_coast_z_fence=self.enable_coast_z_fence,
                coast_z_min=self.coast_z_min,
                coast_z_max=self.coast_z_max,
                hit_residual_min_sq=self.hit_residual_min_sq,
                hit_residual_max_sq=self.hit_residual_max_sq,
                bounce_height_max=self.bounce_height_max,
                bounce_max_coast=self.bounce_max_coast,
                m_hit_target=self.m_hit_target)
        )
        self.next_id += 1
        self._spawn_buffer.append(
            (z_now_arr.copy(), self._frame_counter)
        )

    def _age_spawn_buffer(self):
        """Drop buffer entries older than bootstrap_max_gap_frames."""
        cutoff = self._frame_counter - self.bootstrap_max_gap_frames
        self._spawn_buffer = [
            (z, f) for (z, f) in self._spawn_buffer if f > cutoff
        ]

    def _manage_lifecycle(self):
        active_tracks = []
        for track in self.tracks:
```

```
            # Delete a track that has not updated for too long.
            if track.time_since_update > self.max_age:
                track.state = 'Deleted'

            # Defect 1: ghost track. Tentative (never confirmed) but already
            # coasting past tentative_max_age frames -> noise spawn. Confirmed
            # tracks are untouched.
            elif (track.state == 'Tentative'
                  and track.time_since_update > self.tentative_max_age):
                track.state = 'Deleted'

            # Defect 2: sub-floor coast. Only when the track has no measurement
            # (coasting): with a detection y is measured and no clamp is needed.
            elif (track.time_since_update > 0
                  and track.imm.x[3] < self.floor_kill_y):
                track.state = 'Deleted'

            # Keep the track unless deleted.
            if track.state != 'Deleted':
                active_tracks.append(track)

        self.tracks = active_tracks

        if self.enable_single_ball_nms:
            self._suppress_duplicate_confirmed()

    def _suppress_duplicate_confirmed(self):
        """Single-ball NMS. One ball in play -> at most one Confirmed track. If
```

several exist (phantom duplicates from noise spawns or divergent coasting), keep the best and DELETE the rest.

Deletion (not demotion) is deliberate: a demoted duplicate keeps stealing detections in the Hungarian step, re-confirms and competes again, producing identity switches. Deletion sends detections to the winner; if the winner truly diverges, a fresh detection spawns a new track (slightly higher fragmentation, much smoother trajectory: jerk down, switches 9->3).

Best = (min bucketed-tsu, max hits, min last Mahalanobis^2): actively updated > long-lived > well-fitted.

tsu bucketing: a short coast (tsu <= tsu_tol) counts as 0. Without it one missed frame on a real track (tsu=1) would hand identity to a divergent ghost with tsu=0, causing identity flicker during manoeuvres. With the bucket the real track keeps identity through short turbulence; a ghost with a LONG coast (tsu > tsu_tol) still yields to the active track.”””

```
confirmed = [t for t in self.tracks if t.state == 'Confirmed']
if len(confirmed) < 2:
    return

def _key(t):
    eff_tsu = 0 if t.time_since_update <= self.single_ball_nms_tsu_tol \
        else t.time_since_update
    return (eff_tsu, -t.hits, t._last_mahalanobis_sq)

confirmed.sort(key=_key)
winner = confirmed[0]
```

```
# Merge-identity (fragmentation fix): one ball domain, so all coexisting
# Confirmed tracks are the same physical ball split by a manoeuvre. The
# winner inherits the minimum id of the group so logical identity is
# continuous across the seam. Tracks split by a real trajectory break
# (different rallies) never coexist as Confirmed, so no false merge.
# Carry the max hits too, so the merged track stays a stable NMS winner.
#
# Gate B (merge guard): inherit an id only from physically reconcilable
# members (<= single_ball_merge_max_dist from the winner). A distant FP
# track that reached Confirmed does not donate its id (that would
# legalise a teleport); it is simply deleted as a duplicate.
def _pos(t):
    x = t.imm.x_post
    return np.array([x[0], x[3], x[6]])

wp = _pos(winner)
reconcilable = [
    t for t in confirmed
    if np.linalg.norm(_pos(t) - wp) <= self.single_ball_merge_max_dist
]
winner.track_id = min(t.track_id for t in reconcilable)
winner.hits = max(t.hits for t in reconcilable)

for loser in confirmed[1:]:
    loser.state = 'Deleted'
self.tracks = [t for t in self.tracks if t.state != 'Deleted']
```

```
    def get_confirmed_tracks(self):
        """Return only the tracks we are confident in (Confirmed state)."""
        return [t for t in self.tracks if t.state == 'Confirmed']
```

training_models/model_train_remote.py

```
import os
import torch as t
import torch.nn as nn
import torch.nn.functional as F
from pathlib import Path
from torchvision import transforms
from torchvision.transforms import Compose, ToTensor, PILToTensor, Resize
from tqdm import tqdm
import numpy as np
# import optuna as opt
from ultralytics import YOLO
import albumentations as A


main_model = YOLO('../parsing_datasets_scripts/yolo26m.pt')

custom_transforms = [
    A.MotionBlur(blur_limit=(7, 15), p=0.4),
    A.GaussNoise(std_range=(0.01, 0.05), p=0.2),
    A.CLAHE(clip_limit=4.0, p=0.3),
    A.RandomBrightnessContrast(brightness_limit=0.25,
                               contrast_limit=0.25, p=0.4),
    A.HueSaturationValue(hue_shift_limit=15, sat_shift_limit=30,
```

```
                    val_shift_limit=20, p=0.3),
]

# Purely for server training
main_model.train(data='/workspace/diploma_project/data/server_data.yaml',
                 epochs=100, device='0', imgsz=640,
                 augmentations=custom_transforms, patience=20)

metrics = main_model.val(data='/workspace/diploma_project/data'
                         '/server_data.yaml', imgsz=640)

print(f"mAP50: {metrics.box.map50}")
print(f"mAP50-95: {metrics.box.map}")
```